

FRONTEND CODING ASSESSMENT

Nobleoak Angular Assessment

Current-code walkthrough, architectural decisions, end-to-end request flow, testing strategy and interview preparation.

Repository: [morton-nari/Nobleoak](#)

Architecture: Angular 22 standalone, zoneless, signals and Reactive Forms

Styling: Bootstrap CSS with scoped SCSS; no Bootstrap JavaScript

Prepared: 5 August 2026

This document describes the current implementation only.

Contents

1. Project purpose and technical baseline
2. Application bootstrap and zoneless configuration
3. API layer and local development proxy
4. Journey models, adapter and signal store
5. Dynamic Reactive Form factory
6. Dynamic question rendering
7. Page orchestration and sidebar navigation
8. Quote and additional-question flow
9. Accessibility and responsive styling
10. Testing and verification
11. Limitations and production improvements
12. Interview preparation

1. Project purpose and technical baseline

The application implements the supplied life-insurance journey. **Your Details** is treated as a previously completed, static stage. The assessment begins with the API-provided **Application** pages and finishes with a quote.

Angular 22.1

Standalone

Zoneless

Signals

Reactive Forms

Bootstrap 5.3

SCSS

Vitest

Your Details (static ✓) → Application (About You + Lifestyle) → Review → Quote

The feature structure separates external communication, internal state, forms and presentation:

```
src/app/
├── core/api/                HTTP contracts and service
├── features/quote-journey/
│   ├── components/         Dynamic fields and sidebar
│   ├── data-access/        Adapter and signal store
│   ├── forms/              Dynamic FormRecord factory
│   ├── models/             Internal journey contracts
│   └── pages/              Feature orchestration
```

2. Application bootstrap and zoneless configuration

Standalone startup

```
bootstrapApplication(App, appConfig)
  .catch((err) => console.error(err));
```

`main.ts` bootstraps the standalone root component. There is no `AppModule`. The root component imports and renders `QuoteJourneyPage`, keeping the root shell intentionally small.

Global providers

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideHttpClient(),
  ],
};
```

`provideHttpClient()` makes Angular HTTP infrastructure injectable throughout the application. Global browser error listeners integrate unhandled browser errors with Angular's error handling.

Why zoneless?

Angular 22 is zoneless by default. The project does not install `zone.js` or configure a zone-based provider. Signals, Angular events, Reactive Forms and HTTP notifications explicitly identify state changes.

Decision: use signals for current synchronous UI state and RxJS Observables at the asynchronous HTTP boundary.

Bootstrap styling only

`angular.json` loads Bootstrap's minified CSS before the project's global SCSS. Bootstrap JavaScript is excluded because Angular owns interaction, conditional rendering and DOM state.

3. API layer and development proxy

Typed contracts

`insurance-api.models.ts` describes application pages, questions, answers and the two possible quote responses. Interfaces represent JSON object contracts; type aliases represent unions and reusable type expressions.

```
export type QuoteResponse =  
  | AdditionalQuestionsResponse  
  | QuotedResponse;
```

The literal `status` property creates a discriminated union. Checking `response.status` safely narrows the response to either a final quote or additional pages.

HTTP service responsibility

```
getApplication(): Observable<ApplicationDefinition> {  
  return this.http.get<ApplicationDefinition>('/api/application');  
}  
  
submitQuote(answers: QuoteAnswers): Observable<QuoteResponse> {  
  return this.http.post<QuoteResponse>('/api/quote', { answers });  
}
```

The service constructs typed requests and returns Observables. It does not subscribe, manage loading indicators, build forms or navigate the UI.

Why Network shows localhost

```
Browser: http://127.0.0.1:4200/api/application  
      ↓ Angular development proxy removes /api  
Azure: https://insurance-quote-api-...azurewebsites.net/application
```

`proxy.conf.json` forwards `/api` to the supplied Azure host and rewrites the path. The browser sees a same-origin local request, avoiding the external API's missing CORS headers.

Production note: the Angular proxy works only with `ng serve`. Production hosting must provide an equivalent same-origin rewrite or backend-for-frontend.

4. Journey models, adapter and signal store

Internal model

```
export interface JourneySection {
  readonly id: string;
  readonly title: string;
  readonly questions: readonly ApplicationQuestion[];
}
```

The current adapter preserves the API's About You and Lifestyle groupings, removes empty pages and creates a boundary between external API contracts and internal UI terminology. It does not move email or phone into Your Details.

Private writable, public read-only

```
private readonly journeyState = signal<JourneyDefinition | null>(null);
readonly journey = this.journeyState.asReadOnly();
readonly sections = computed(() => this.journeyState()?.sections ?? []);
```

Only the store mutates state. Components call store methods and consume read-only signals, preventing arbitrary UI code from creating inconsistent states.

Construct	Purpose	Example
<code>signal()</code>	Explicitly mutable state	Loading status, quote, active index
<code>computed()</code>	Derived state	Sections and active section
<code>asReadOnly()</code>	Safe public state	Components can read but not set
Observable	Asynchronous operation	GET and POST responses

Loading state machine

```
idle → loading → loaded
      ↘ error
```

`loadApplication()` resets stale state, calls the service and converts the first HTTP result into signals.

`take(1)` communicates the single-response expectation; `takeUntilDestroyed()` protects the subscription lifecycle.

Navigation state

`goToSection(index)` rejects non-integer and out-of-range indexes before updating the active section. In the current UI, section state controls the sidebar highlight and focus; it does not hide the initially available API questions.

5. Dynamic Reactive Form factory

Why FormRecord?

```
export type JourneyFormControl = FormControl<string | number | null>;
export type JourneyForm = FormRecord<JourneyFormControl>;
```

A `FormGroup` is ideal when keys are known statically. Here, control IDs come from the API and additional IDs may arrive after submission, so `FormRecord` represents a dynamic string-keyed control collection.

Creation

Journey sections → flatten questions → one `FormControl` per question ID → `FormRecord`

The factory uses API metadata to apply `required`, email and minimum-zero validation. It adds a question only when its ID is not already present, preserving existing control instances, values and touched state.

Answer extraction

`getAnswers()` reads the form's raw value, removes `null` and empty-string values, and returns the flat object required by the quote service.

```
{
  email: 'person@example.com',
  phone: '0412345678',
  occupation: 'Teacher',
  smokedLast12Months: 'No'
}
```

Decision: Reactive Forms manage field values, validation, touched state and native value accessors; signals manage journey and UI state.

6. Dynamic question rendering

`DynamicQuestion` receives two required signal inputs:

```
readonly question = input.required<ApplicationQuestion>();
readonly control = input.required<JourneyFormControl>();
```

The component renders native HTML according to `question.type`:

API type	Rendered control	Special behavior
email	<code><input type="email"></code>	Email validator and autofill
text	<code><input type="text"></code>	Phone gets tel input mode/autofill
select	<code><select></code>	Options rendered from API
radio	<code><fieldset></code> and radios	Native group semantics
number	<code><input type="number"></code>	Number value accessor, min 0, step 1

Computed IDs connect labels, inputs and errors. Validation appears only when a control is touched and invalid. Native controls retain browser keyboard, autofill and assistive-technology behavior.

Adding fields

A new field using an existing type is automatically preserved by the adapter, added by the factory, rendered by the component and included in the payload. A genuinely new type requires updating the type union, renderer, validators and tests.

7. Page orchestration and sidebar navigation

Page responsibility

`QuoteJourneyPage` coordinates the store, form factory and presentation. Local signals hold the form, completed section IDs, initial section count and review visibility.

An Angular `effect` creates the form after the journey signal receives the GET result. When the same journey later gains additional pages, the effect calls `addQuestions()` instead of recreating the form.

All initial fields are visible

The initial form renders both API sections together:

```
About You: email, phone, occupation  
Lifestyle: smokedLast12Months
```

About You is active initially. Changing an About You field activates About You; selecting Yes or No activates Lifestyle. The section index controls progress styling, not page routing.

Modern component communication

```
readonly sections = input.required<readonly JourneySection[]>();  
readonly sectionSelected = output<number>();
```

Inputs are read-only signal inputs that represent current parent state. The output is an event emitter, not stored state. The sidebar emits a selection; the page validates the request through the store and passes the updated index back down.

Why no Router?

The journey is one temporary form workflow without deep links, route restoration or browser-history requirements. Signals provide simpler local navigation. Router should be introduced if independently addressable URLs become a requirement.

8. Quote and additional-question flow

Review and request

Next validates every visible section, marks controls touched, activates the first invalid section and focuses the first invalid control. Valid sections lead to the review screen, which reads values directly from the same `FormRecord`.

```
Form → validate → review → getAnswers() → store.submitQuote() → POST /quote
```

Duplicate protection

The button is disabled while submitting, and the store independently rejects a second call when submission status is already `submitting`.

Non-smoker

A `quoted` response writes the quote signal and renders Life Protect, A\$500,000 cover and the returned monthly premium.

Smoker follow-up

An `additionalQuestionsRequired` response is normalized and appended immutably. Duplicate section IDs are ignored. The active index moves to the first new section.

Existing FormRecord	Same FormRecord after response
— email	— email (preserved)
— phone	— phone (preserved)
— occupation	— occupation (preserved)
— smokedLast12Months	— smokedLast12Months (preserved)
	— cigarettesPerWeek (new)

The follow-up view shows only Smoking Details, but the second POST includes every accumulated answer.

9. Accessibility and responsive styling

- Semantic landmarks: header, main, aside, nav and section.
- A keyboard-visible skip link targets `#journey-content`.
- Native labels, fieldsets, legends, required state and linked errors.
- `aria-invalid`, `aria-describedby`, status and alert regions.
- Programmatic heading focus after major view transitions.
- Email/telephone autocomplete and appropriate mobile input modes.
- Bootstrap responsive grid: sidebar and content columns on desktop, stacked on smaller screens.
- Sticky sidebar only at desktop widths.

Scoped SCSS uses Bootstrap variables and neutral active styling to match the wireframe. Bootstrap button variables change the primary button to the required dark scheme without rewriting the component.

Verification: Lighthouse accessibility 100, no horizontal overflow, and no application console/runtime/network errors during live Chrome walkthroughs.

10. Testing and verification

Suite	What it proves
API service	Correct GET/POST URL, method, request wrapper and response types
Adapter	API page grouping is preserved and empty pages are removed
Store	Loading, errors, section selection, submission, quote and appended-page state
Form factory	Dynamic controls, validators, answer preservation and payload extraction
Dynamic question	Native control choice, options, values and accessible semantics
Page	Complete validation, review, retry, non-smoker and smoker workflows
Root	Standalone application composition

Current automated result: 7 test files and 29 passing tests. `npm run build` also verifies strict TypeScript, Angular templates, component styles and production bundling.

Live Chrome verification exercised both real API paths: A\$64.85 for the non-smoker example and A\$104.75 after the smoker follow-up.

11. Limitations and production improvements

Current limitation	Production direction
Development-only proxy	Hosting rewrite or backend-for-frontend for <code>/api</code>
Compile-time API types only	Runtime schema validation with a type guard or schema library
In-memory form state	Confirm security requirements before adding draft persistence
Five supported question types	Extend union, renderer, validators and tests as the contract grows
No URL state	Add Router only for deep links/history/refresh restoration
No committed browser E2E suite	Add Playwright for critical flows and responsive/keyboard checks
No runtime authentication	Add auth only when supplied by the backend contract

Sensitive answers are not logged. A production design would also require an explicit policy for PII, monitoring redaction, server-side validation, authentication, authorization and rate limiting.

12. Interview preparation

Give a 60-second overview of the architecture.

The standalone zoneless Angular application calls a typed API service through a local same-origin proxy. A feature store converts HTTP Observables into private writable and public read-only signals. API pages are normalized into journey sections. A dynamic FormRecord factory creates controls and validators from question metadata, while a generic component renders native controls. The page coordinates validation, review, sidebar state and dynamically appended follow-up questions.

Why signals?

They suit current synchronous UI state and integrate directly with zoneless Angular templates. Computed signals derive sections and active state without duplicated writable values. RxJS remains appropriate for asynchronous HTTP operations.

Why not NgRx?

The feature has a small state model with limited side effects. A signal-based service provides encapsulation without reducers, actions and effects. NgRx would be reconsidered for complex cross-feature state, event auditing or many coordinated side effects.

Why Reactive Forms rather than signals for fields?

Reactive Forms already provide validators, touched/invalid state, native value accessors and dynamic control APIs. Signals manage journey state; Reactive Forms are the specialized field-state solution.

Why FormRecord rather than FormGroup?

The backend supplies control IDs at runtime and may add more after submission. FormRecord represents a consistently typed, dynamic string-keyed collection. FormGroup is preferable when keys are known statically.

What is the difference between `signal`, `computed`, `input` and `output`?

A writable signal stores mutable state. A computed signal derives a value from other signals.

`input.required()` creates a read-only signal input controlled by the parent. `output()` emits events and does not retain state.

Why are store signals private?

Only store methods should perform state transitions. Public read-only signals prevent components from setting impossible loading or submission combinations.

Why keep the adapter when the shapes are similar?

It protects the UI from external naming and normalization changes, removes empty pages and gives initial and additional pages one consistent internal representation.

Why no Angular Router?

The design is one form workflow without deep links, independent URLs or browser-history requirements. Signal state avoids unnecessary URL/form synchronization. Router would be added if those requirements appeared.

How does the proxy work?

The browser calls local `/api`. Angular's development server matches the prefix, removes it and forwards the request to the supplied Azure host. The browser sees localhost because the forwarding happens server-side, avoiding CORS.

How are additional questions handled?

The discriminated quote response indicates additional pages. The store appends unique sections immutably. A page effect adds only missing FormControls to the existing FormRecord, so previous answers survive and the next POST includes everything.

How would you add a field?

An existing supported type requires no page change: the adapter preserves it, the factory creates it, the renderer displays it and answer extraction submits it. A new type requires extending the type union, renderer, validators and tests.

How do you prevent duplicate submissions?

The button is disabled during submission and the store rejects calls while its submission state is already `submitting`.

How did you test it?

HTTP tests verify request contracts, pure tests verify adapters and form factories, Subject-backed store tests control intermediate states, and page integration tests cover both quote paths and retries. Production build, Lighthouse and live Chrome/API checks provide additional verification.

What would you improve with more time?

Add runtime API validation, Playwright end-to-end coverage, a production proxy/BFF, and persistence only after confirming PII security requirements. I would also perform a formal screen-reader pass and add redacted production monitoring.

Questions to ask the assessor

- Should individual journey states be deep-linkable or restorable after refresh?
- Is the API schema versioned, and can new question types appear without a frontend release?
- What are the persistence and retention requirements for personal information?
- What production host will own the same-origin API forwarding rule?
- Are there established design-system and accessibility test requirements?

Final demonstration checklist

1. Run `npm install`, `npm test`, `npm run build` and `npm start`.

2. Show the GET response in Network and explain the proxy.
3. Submit an empty form to demonstrate validation and focus.
4. Complete the non-smoker flow and show the request payload and quote.
5. Complete the smoker flow and show the appended question and accumulated second payload.
6. Resize to mobile width and demonstrate the stacked layout.
7. Explain the store, FormRecord and discriminated union from the source.

End of walkthrough — Nobleoak Angular Assessment